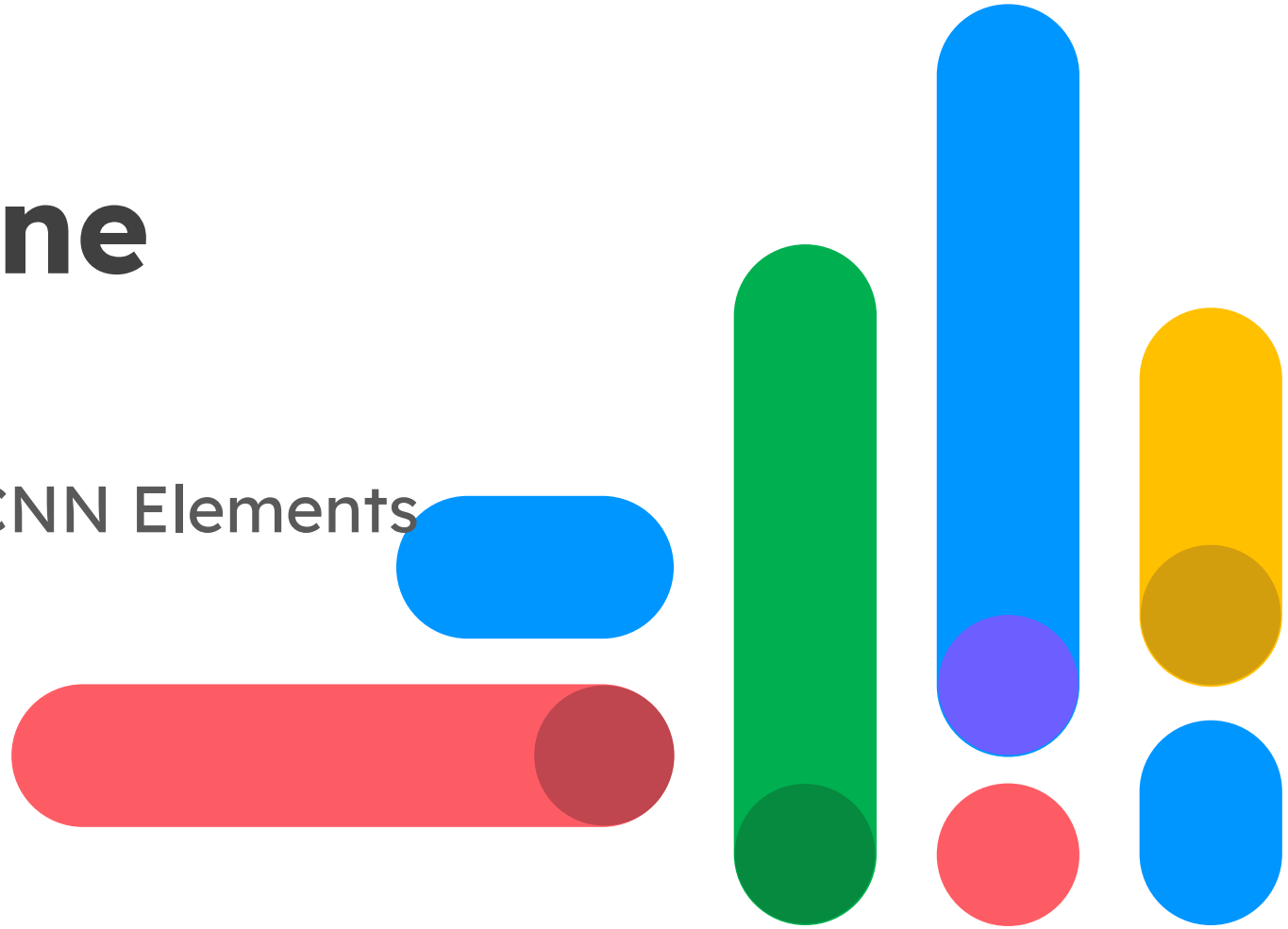
Deep Learning

# Applied Machine Learning

## Computer Visions :: Basis & CNN Elements

Assistant Professor | **Chun-Hsiang Chan**

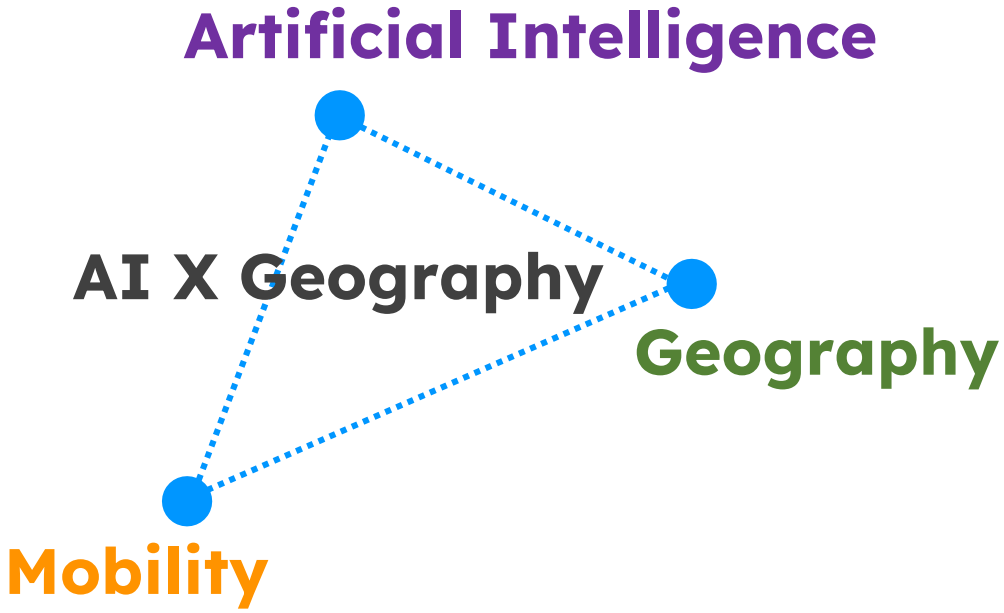
Department of Geography
National Taiwan Normal University



# Outline

#GeoAI #Mobility #Geography #STDS


01 What is Computer Vision?
02 Images as Numerical Data
03 Convolution
04 Kernels to Filters
05 Stride & Padding
06 Pooling
07 CNN Architect
08 References

# What is Computer Vision

**Computer Vision** is a field of AI that enables computers to see, identify, and understand visual information from images and videos, similar to how humans see and process this information. It uses machine learning to conduct tasks like recognizing objects, detecting faces, reading text, or tracking movement (e.g., **prediction**, **estimation**, **detection**, and **localization**). The visual data format may rely on a range of visual signals sourced from, e.g., medical scanning machines, video frames, images taken by a single camera, a scene sourced from multiple cameras, and multimodal data related to natural language.

Source: https://hai.stanford.edu/ai-definitions/what-is-computer-vision
Zou, L. (2022). *Meta-learning: theory, algorithms and applications*. Elsevier.

# What is Computer Vision

## Classification Problem :: X=image; y=class

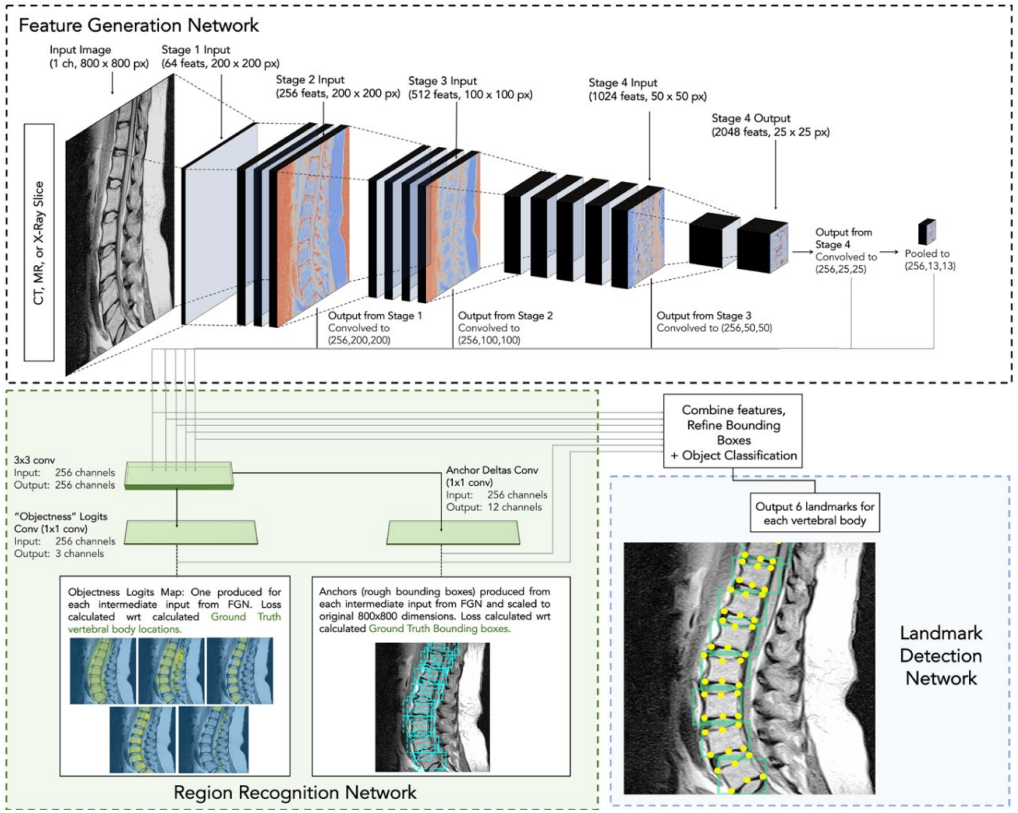


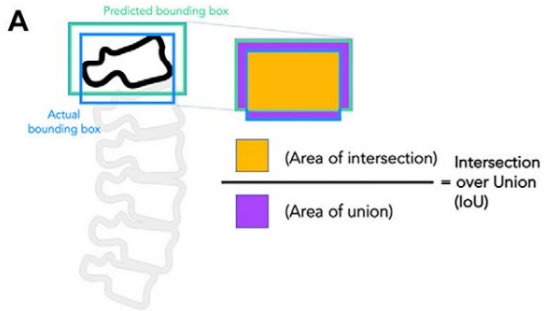


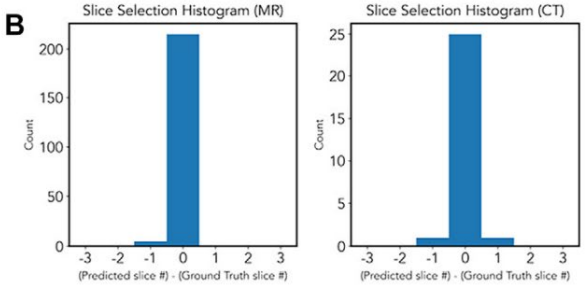


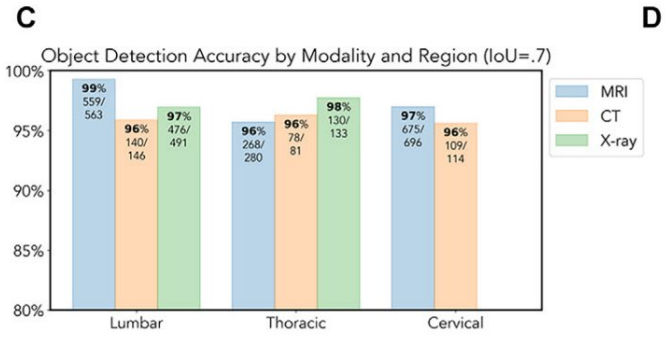
# What is Computer Vision

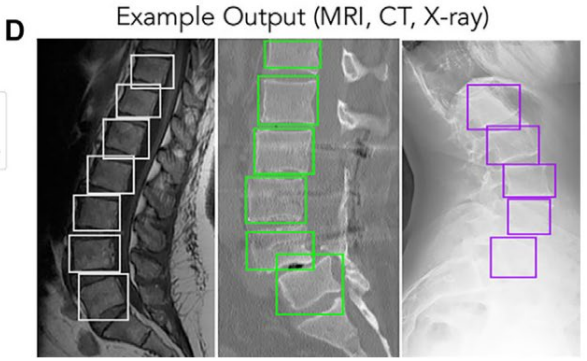
## Detection & Localization :: y=(class, x,y,w,h)



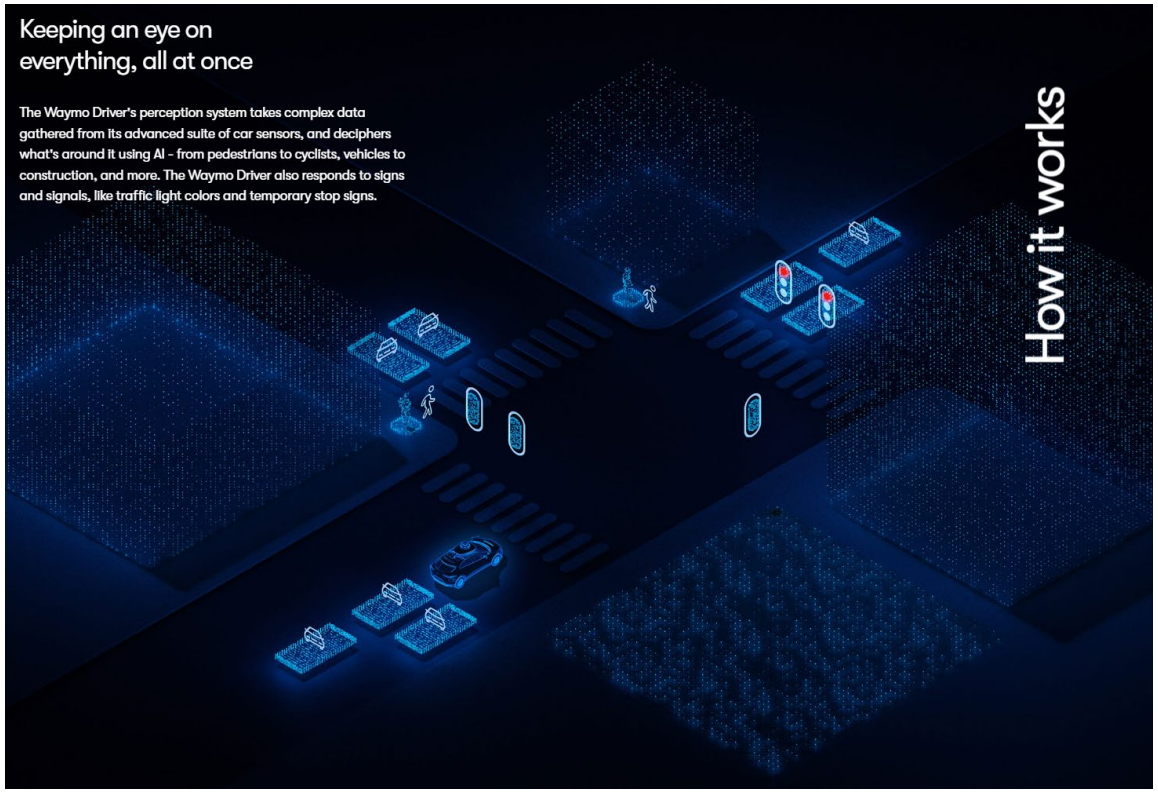




Suri, Abhinav, et al. "Vertebral deformity measurements at MRI, CT, and radiography using deep learning." Radiology: Artificial Intelligence 4.1 (2021): e210015.

# What is Computer Vision

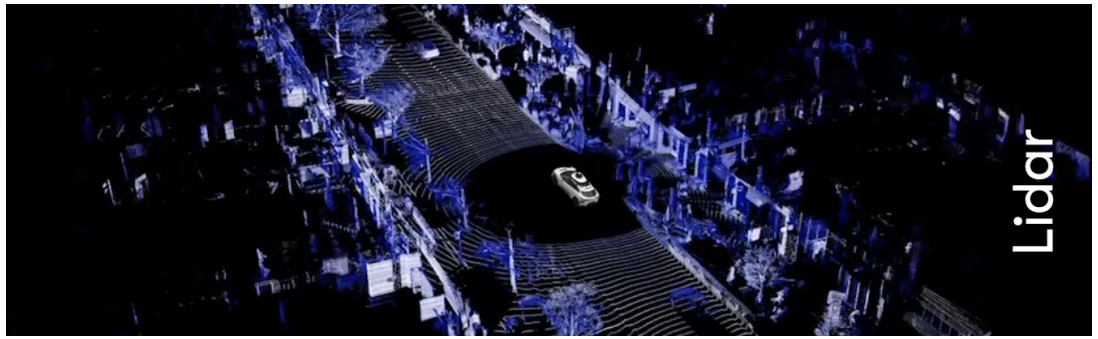


## Detection & Localization :: f(x,y) ➔ class





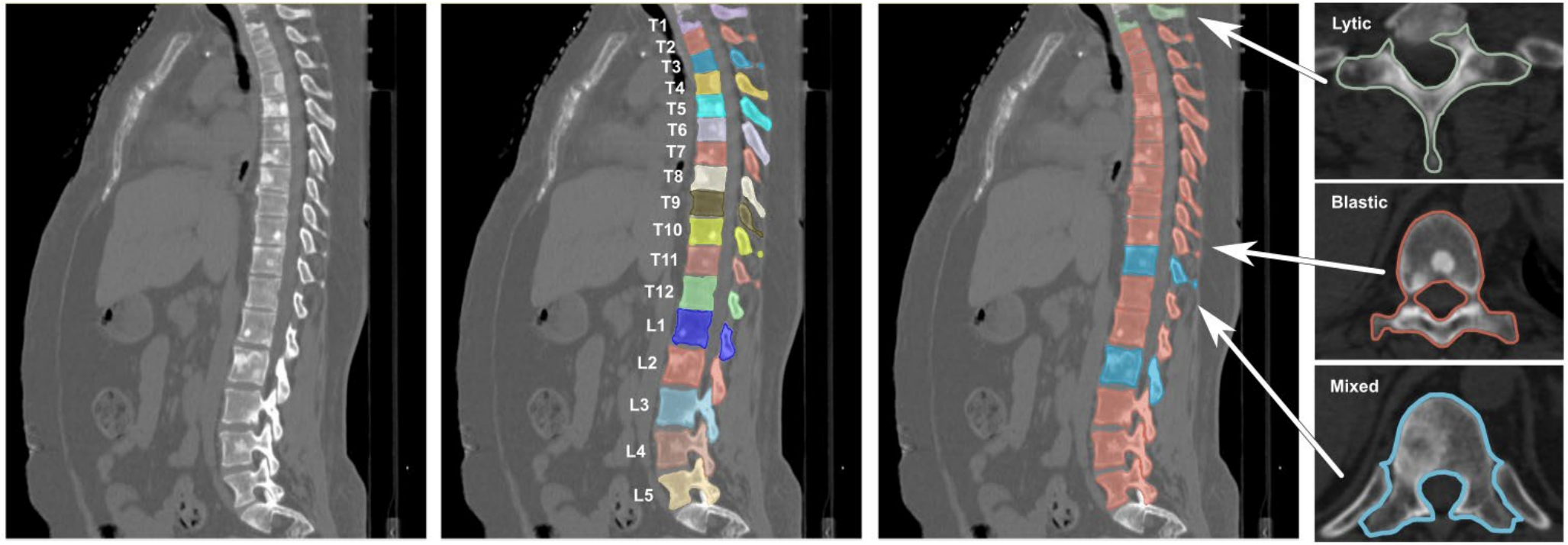





Source: https://waymo.com/waymo-driver/

# What is Computer Vision

## Detection & Localization :: f(x,y) ➔ class w indices




Source: https://www.cancerimagingarchive.net/collection/spine-mets-ct-seg/

# What is Computer Vision

**Computer Vision Levels ($X = image$)**

Image Class:

$$y = class$$

Object Detection:

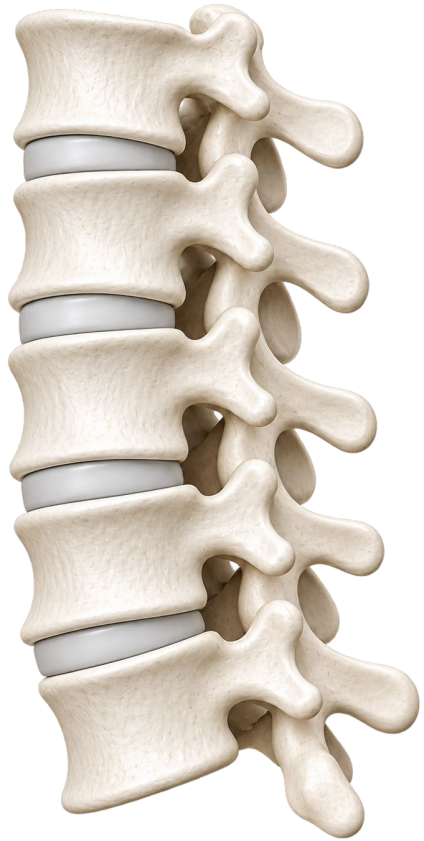
$$y = (class, x, y, w, h)$$

Sematic Segmentation:

$$f(x, y) = class$$

Instance Segmentation:

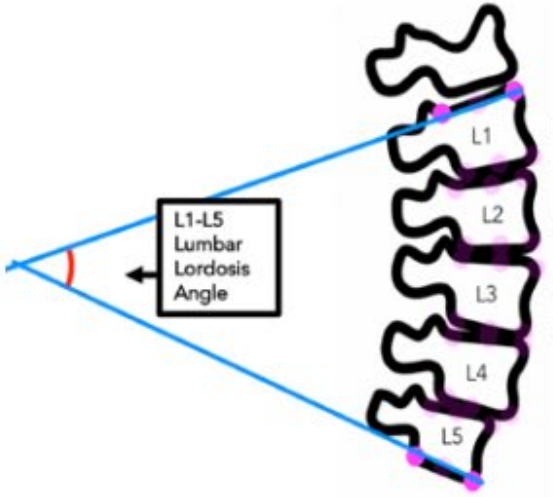
$$f(x, y) = class_idx$$

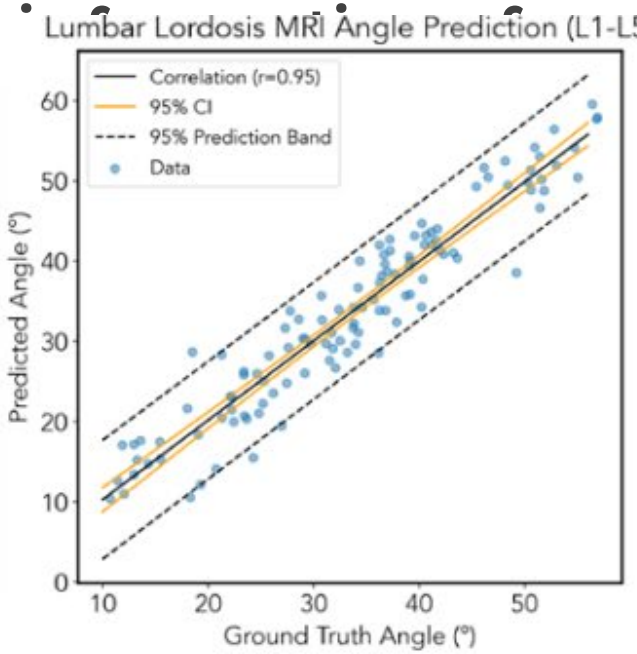


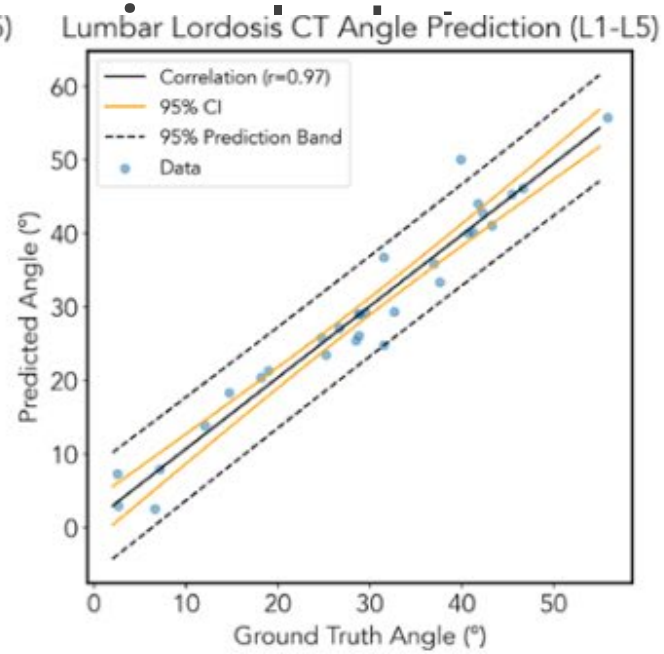
Zou, L. (2022). *Meta-learning: theory, algorithms and applications*. Elsevier.

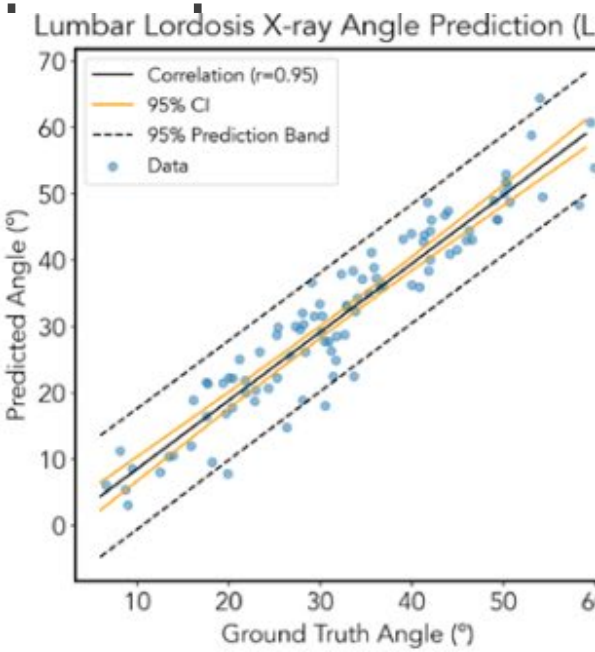
# What is Computer Vision

Computer Vision aims to enable machines to extract




Zou, L. (2022). *Meta-learning: theory, algorithms and applications*. Elsevier.

# Image as Numerical Data

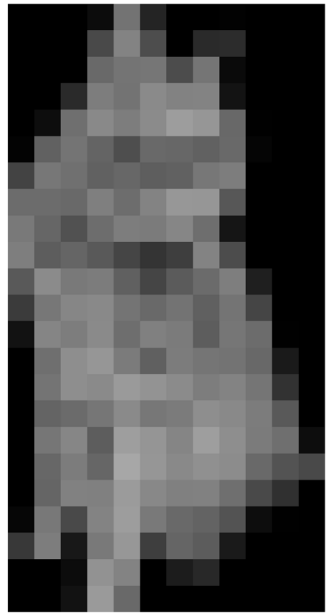
For a **grayscale** image, …

This is what you see …



This is what your computer sees …



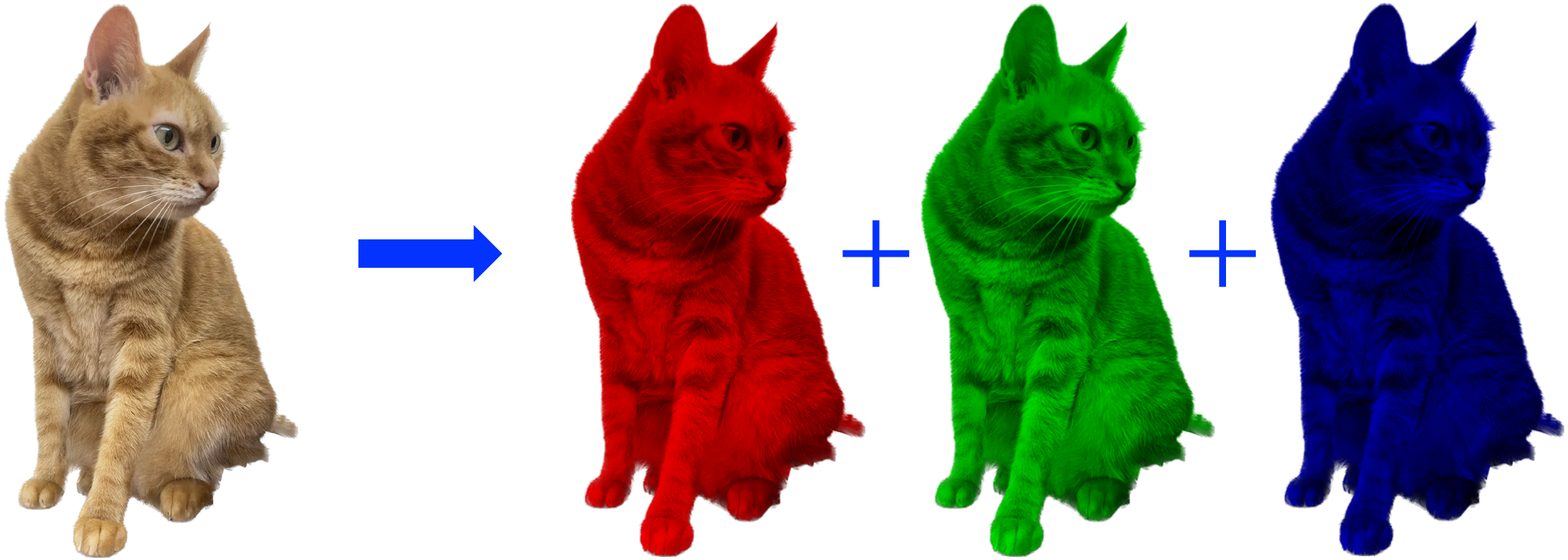

Pixel-value matrix
Each cell = one pixel in the reduced image

| Row | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 12 | 115 | 36 | 0 | 0 | 2 | 0 | 0 | 0 |
| 2 | 0 | 0 | 0 | 74 | 131 | 77 | 0 | 42 | 44 | 0 | 0 | 0 |
| 3 | 0 | 0 | 0 | 106 | 116 | 118 | 76 | 117 | 10 | 0 | 0 | 0 |
| 4 | 0 | 0 | 42 | 125 | 115 | 138 | 130 | 132 | 19 | 0 | 0 | 0 |
| 5 | 0 | 13 | 111 | 137 | 125 | 139 | 157 | 150 | 104 | 1 | 0 | 0 |
| 6 | 3 | 97 | 116 | 100 | 78 | 105 | 103 | 97 | 105 | 5 | 0 | 0 |
| 7 | 67 | 119 | 112 | 97 | 103 | 94 | 97 | 119 | 125 | 0 | 0 | 0 |
| 8 | 108 | 108 | 105 | 127 | 109 | 132 | 151 | 150 | 87 | 0 | 0 | 0 |
| 9 | 121 | 102 | 82 | 109 | 125 | 123 | 134 | 110 | 21 | 0 | 0 | 0 |
| 10 | 126 | 93 | 101 | 90 | 69 | 53 | 62 | 132 | 76 | 0 | 0 | 0 |
| 11 | 89 | 139 | 122 | 124 | 96 | 69 | 91 | 107 | 126 | 31 | 0 | 0 |
| 12 | 59 | 119 | 134 | 137 | 116 | 105 | 115 | 96 | 116 | 69 | 0 | 0 |
| 13 | 18 | 133 | 125 | 138 | 110 | 128 | 123 | 93 | 118 | 108 | 1 | 0 |
| 14 | 0 | 111 | 142 | 150 | 123 | 96 | 124 | 117 | 115 | 104 | 26 | 0 |
| 15 | 0 | 116 | 142 | 138 | 154 | 147 | 139 | 125 | 131 | 117 | 50 | 0 |
| 16 | 0 | 100 | 107 | 118 | 138 | 119 | 123 | 142 | 137 | 124 | 90 | 0 |
| 17 | 0 | 118 | 134 | 93 | 158 | 149 | 133 | 158 | 142 | 123 | 111 | 13 |
| 18 | 0 | 103 | 124 | 102 | 167 | 150 | 137 | 144 | 140 | 106 | 83 | 73 |
| 19 | 0 | 101 | 129 | 128 | 155 | 136 | 128 | 129 | 111 | 73 | 48 | 0 |
| 20 | 6 | 120 | 74 | 131 | 157 | 122 | 106 | 99 | 83 | 13 | 1 | 0 |
| 21 | 57 | 125 | 24 | 122 | 151 | 62 | 101 | 92 | 26 | 0 | 0 | 0 |
| 22 | 0 | 0 | 12 | 146 | 131 | 0 | 28 | 41 | 0 | 0 | 0 | 0 |
| 23 | 0 | 0 | 18 | 154 | 105 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Column

# Image as Numerical Data

For a **RGB** image, …



# Image as Numerical Data

**Image to Tensor**

Given 32 RGB images with a size of 200 x 300 pixels.

Conventional expression

$$32 \times 200 \times 300 \times 3$$

$$Batch \times Height \times width \times channels$$

PyTorch

$$N \times C \times H \times W$$

Tensorflow

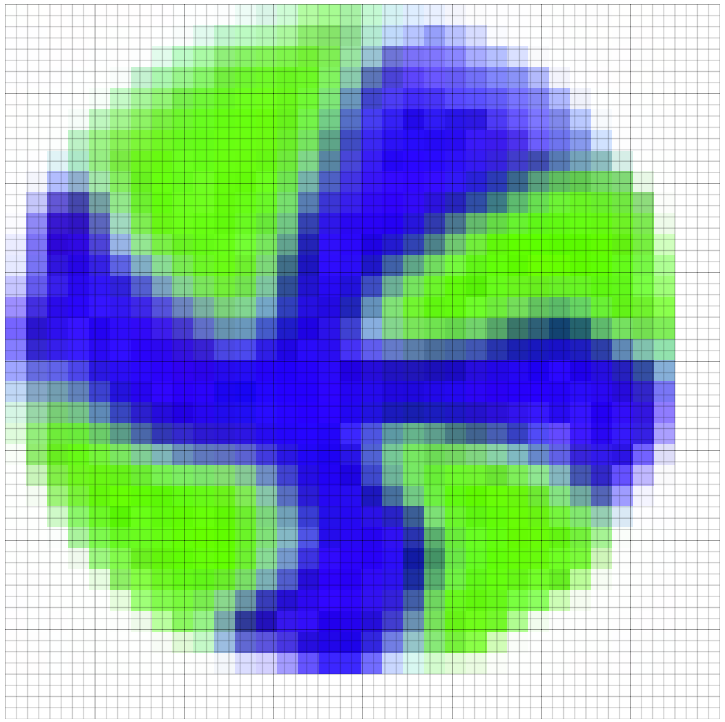
$$N \times H \times W \times C$$

# Convolution

**Why do we need Convolution?**

If we conduct fully connected network for the image, then that will be a disaster due to the high dimension.

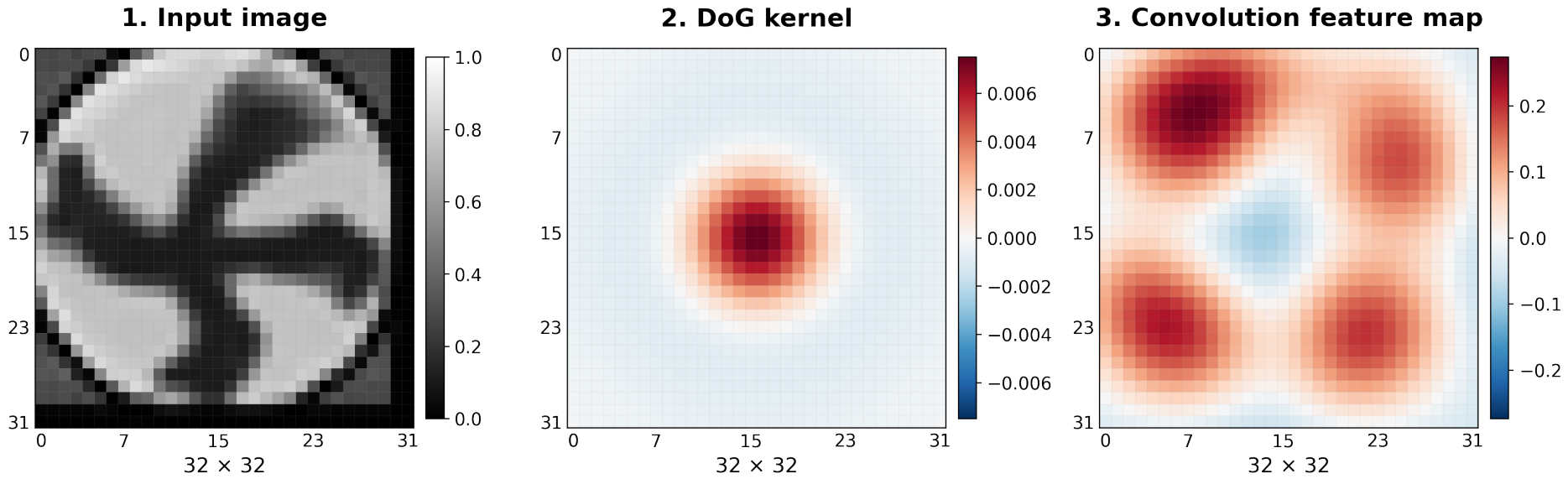
Given a $200 \times 200$ RGB image, the dimension of input feature will be $200 \times 200 \times 3 = 120{,}000$, and if we have 1000 neurons, then a total number of weights for model training will be $120{,}000 \times 1000 = 120{,}000{,}000$, which is a heavy computation load.



# Convolution

In most situations, a nearer pixel has higher correlation than a further pixel, just like spatial proximity effect.

As a result, we may use a deeper layer to store high-level features from an image.



# Convolution

**Extract Local Features**

Uses learnable filters (kernels) to slide and scan across the entire image. Through element-wise multiplication and summation.

**Weight Sharing**

The same filter is applied to different positions across the entire image. This significantly reduces the number of parameters that need to be learned, prevents overfitting, and provides translation invariance.

# Convolution

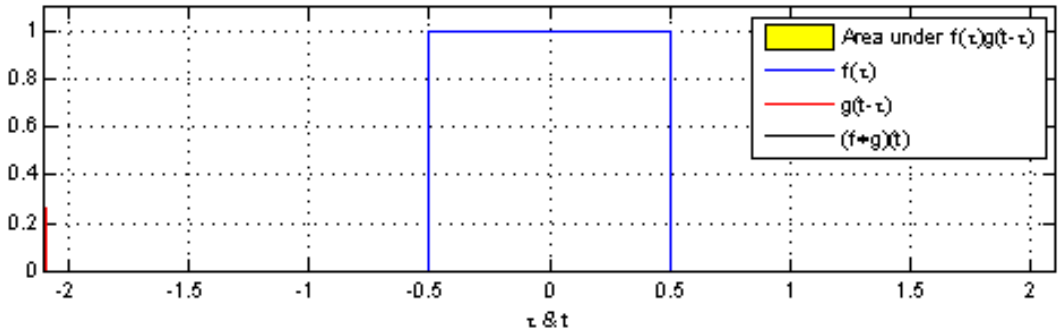
## Preserving Spatial Structure

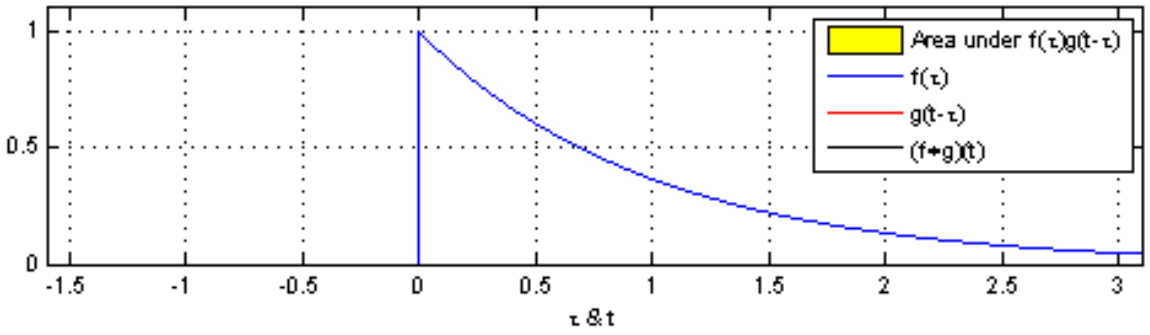
Traditional Fully Connected Neural Networks (MLPs) flatten images into one-dimensional vectors, destroying the spatial relationships between pixels. In contrast, convolution operations fully preserve the 2D/3D spatial and channel structure of the input data.

# Convolution

**Convolution** is a mathematical operation on two functions $f$ and $g$ that produces a third function, $f * g$, as the integral of the product of the two functions after one is reflected about the y-axis and shifted.

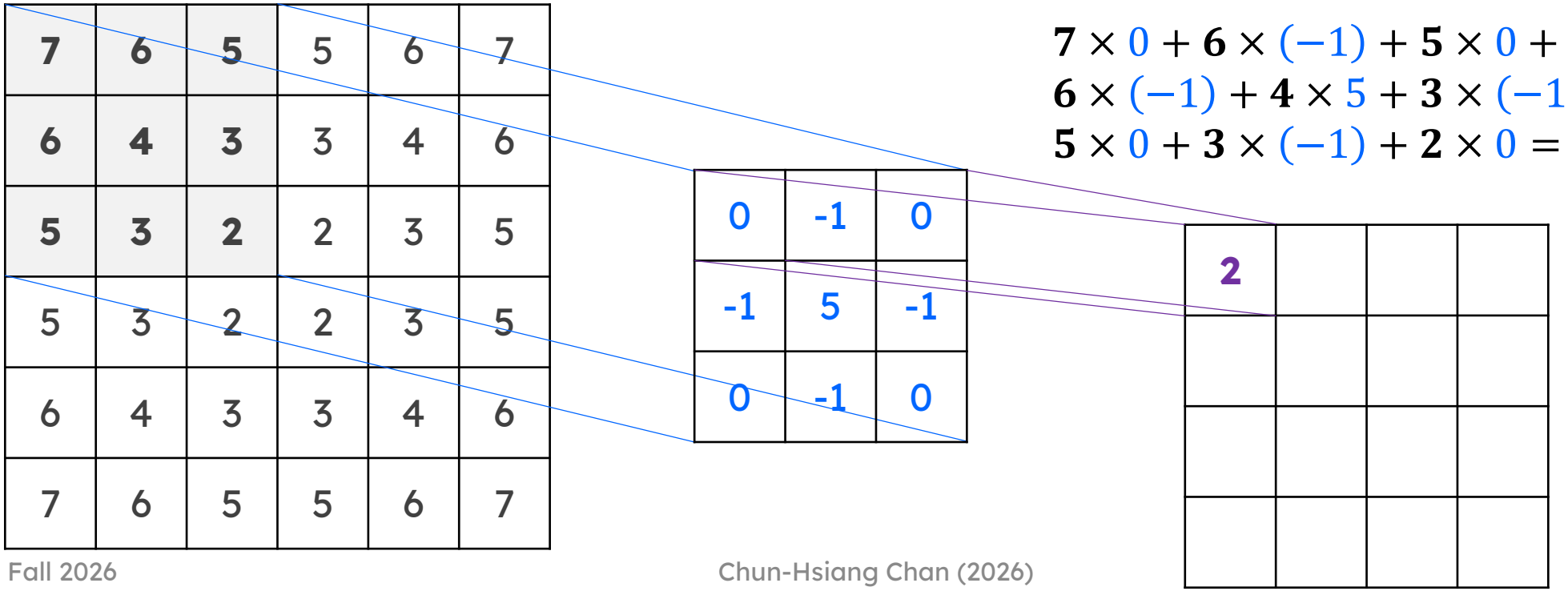
$$(f * g)(t) := \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau$$



# Convolution

From 1D convolution to 2D convolution, we may extract the feature characteristics from the input. In CNN, we adopt **cross-correlation** to compute 2D convolution via sliding window, element-wise multiplication, and summation.



# Convolution

Compute all cells …

**2D Convolution (CNN cross-correlation): stride = 1, no padding**

**Input**

| | | | | | |
|---|---|---|---|---|---|
| 7 | 6 | 5 | 5 | 6 | 7 |
| 6 | 4 | 3 | 3 | 4 | 6 |
| 5 | 3 | 2 | 2 | 3 | 5 |
| 5 | 3 | 2 | 2 | 3 | 5 |
| 6 | 4 | 3 | 3 | 4 | 6 |
| 7 | 6 | 5 | 5 | 6 | 7 |

**3 × 3 kernel**

| | | |
|---|---|---|
| 0 | -1 | 0 |
| -1 | 5 | -1 |
| 0 | -1 | 0 |

**Output**

| | | | |
|---|---|---|---|
| 2 | | | |
| | | | |
| | | | |
| | | | |

**Step 1: output[0,0]**

$7\times0 + 6\times(-1) + 5\times0 + 6\times(-1) + 4\times5 + 3\times(-1) + 5\times0 + 3\times(-1) + 2\times0 = 2$

# Convolution

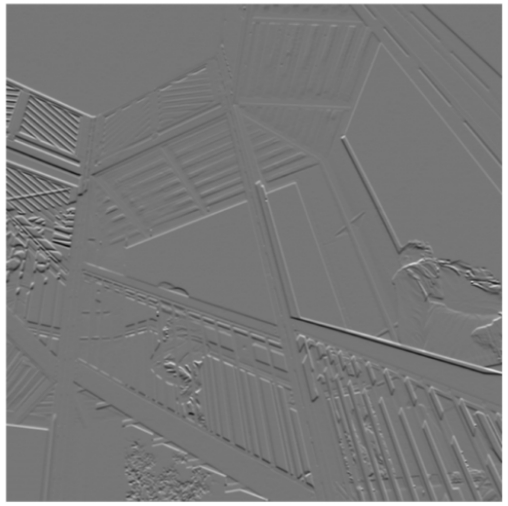
## Some Applications :: from scipy import ndimage, datasets

```
ascent = datasets.ascent().astype('int32')
sobel_h = ndimage.sobel(ascent, 0)  # horizontal
gradientsobel_v = ndimage.sobel(ascent, 1)  # vertical
gradientmagnitude = np.sqrt(sobel_h**2 + sobel_v**2)
magnitude *= 255.0 / np.max(magnitude)  # normalization
```

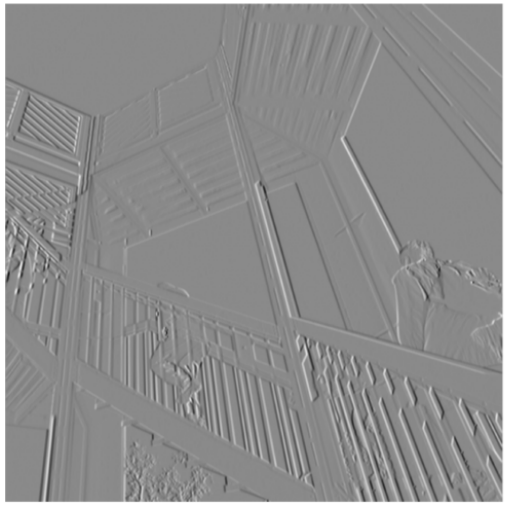
original



horizontal



vertical



magnitude



# Convolution

Given a RGB image ($100 \times 100 \times 3$), how can we do convolution and what is the dimension of the result?

Input: $100 \times 100 \times 3$

Filter: $3 \times 3 \times 3$ ➔ $K_R, K_G, K_B$

Stride: 1; Padding: 1; Number of Filter: 1

$$O = \left[\frac{N + 2P - F}{S}\right] + 1$$

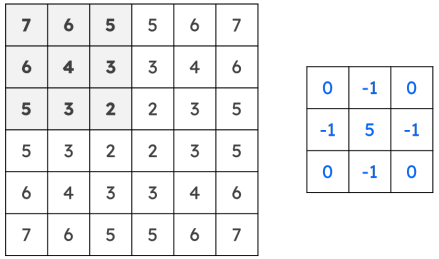
---

Feature map: $Y(i,j) = X_R * K_R + X_G * K_G + X_B * K_B$

Dimension: $98 \times 98 \times 1$

64 Filters ➔ $98 \times 98 \times \mathbf{64}$

# Kernels to Filters



| | | | | | |
|---|---|---|---|---|---|
| 7 | 6 | 5 | 5 | 6 | 7 |
| 6 | 4 | 3 | 3 | 4 | 6 |
| 5 | 3 | 2 | 2 | 3 | 5 |
| 5 | 3 | 2 | 2 | 3 | 5 |
| 6 | 4 | 3 | 3 | 4 | 6 |
| 7 | 6 | 5 | 5 | 6 | 7 |



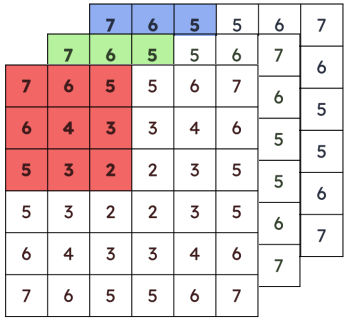
| | | |
|---|---|---|
| 0 | -1 | 0 |
| -1 | 5 | -1 |
| 0 | -1 | 0 |

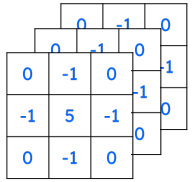
## Kernels

A Kernel is a small 2D matrix (e.g., $3 \times 3$ or $5 \times 5$) containing a set of "learnable weights."

It serves as the core element of a convolutional layer. The Kernel slides across the input image, performing **element-wise multiplication and summation** with the corresponding pixels to detect specific features (such as horizontal edges, vertical lines, etc.).

# Kernels to Filters




## Filters

While Kernel and Filter are often used interchangeably in casual discussions, strictly speaking, they differ when dealing with multi-channel inputs.

After performing convolution, a Filter combines information from all channels to produce a single Feature Map. If a convolutional layer has 64 Filters, it will output 64 distinct Feature Maps.

# Strides & Padding

**Strides**

The number of pixels (or "steps") the Kernel/Filter moves as it slides across the image.

**Stride = 1**

The Kernel moves 1 pixel at a time, resulting in a larger Feature Map that retains maximum fine detail.

**Stride = 2**

The Kernel moves 2 pixels at a time, reducing the output dimensions by roughly half. This achieves a downsampling effect, which reduces computational load and enlarges the receptive field.

# Strides & Padding

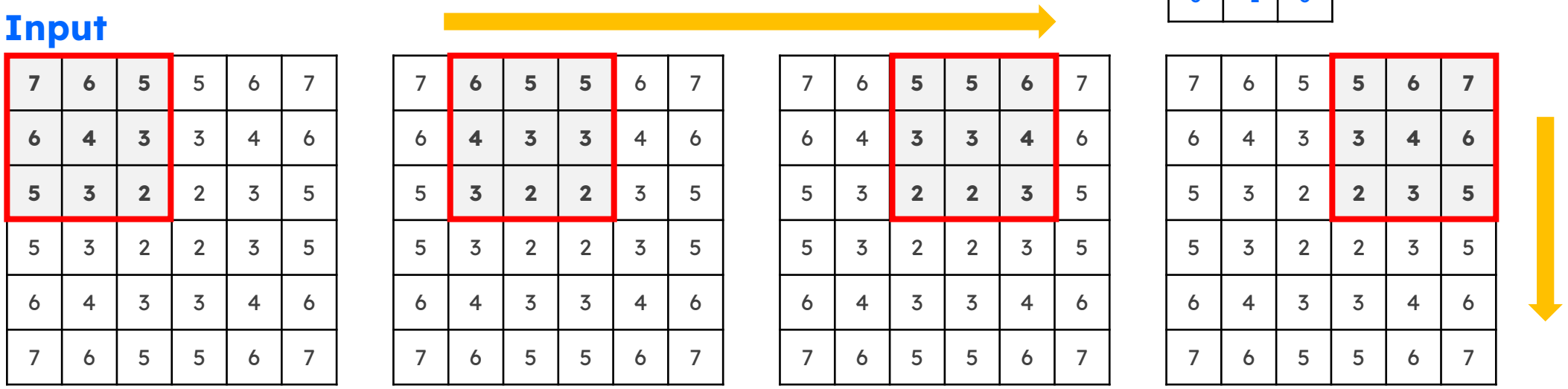
## Strides Demo

Given stride=1, …

**Filter**

| | | |
|---|---|---|
| 0 | -1 | 0 |
| -1 | 5 | -1 |
| 0 | -1 | 0 |

**Input**



# Strides & Padding

## Strides Demo

Given stride=2, ...

**Filter**

| | | |
|---|---|---|
| 0 | -1 | 0 |
| -1 | 5 | -1 |
| 0 | -1 | 0 |

**Input**

| | | | | | |
|---|---|---|---|---|---|
| **7** | **6** | **5** | 5 | 6 | 7 |
| **6** | **4** | **3** | 3 | 4 | 6 |
| **5** | **3** | **2** | 2 | 3 | 5 |
| 5 | 3 | 2 | 2 | 3 | 5 |
| 6 | 4 | 3 | 3 | 4 | 6 |
| 7 | 6 | 5 | 5 | 6 | 7 |

| | | | | | |
|---|---|---|---|---|---|
| 7 | 6 | **5** | **5** | **6** | 7 |
| 6 | 4 | **3** | **3** | **4** | 6 |
| 5 | 3 | **2** | **2** | **3** | 5 |
| 5 | 3 | 2 | 2 | 3 | 5 |
| 6 | 4 | 3 | 3 | 4 | 6 |
| 7 | 6 | 5 | 5 | 6 | 7 |

| | | | | | |
|---|---|---|---|---|---|
| 7 | 6 | 5 | 5 | 6 | 7 |
| 6 | 4 | 3 | 3 | 4 | 6 |
| **5** | **3** | **2** | 2 | 3 | 5 |
| **5** | **3** | **2** | 2 | 3 | 5 |
| **6** | **4** | **3** | 3 | 4 | 6 |
| 7 | 6 | 5 | 5 | 6 | 7 |

| | | | | | |
|---|---|---|---|---|---|
| 7 | 6 | 5 | 5 | 6 | 7 |
| 6 | 4 | 3 | 3 | 4 | 6 |
| 5 | 3 | **2** | **2** | **3** | 5 |
| 5 | 3 | **2** | **2** | **3** | 5 |
| 6 | 4 | **3** | **3** | **4** | 6 |
| 7 | 6 | 5 | 5 | 6 | 7 |

# Strides & Padding

**Padding**
Adding extra border pixels (typically filled with zeros, hence often called Zero-padding) around the edges of the input image or Feature Map.
**Prevents rapid dimension shrinking**
Without padding, image dimensions shrink after every convolution layer because edge pixels are scanned fewer times.
**Preserves edge information**
It enables pixels near the corners and edges to be scanned multiple times, preventing valuable edge features from being lost in deeper layers.

# Strides & Padding

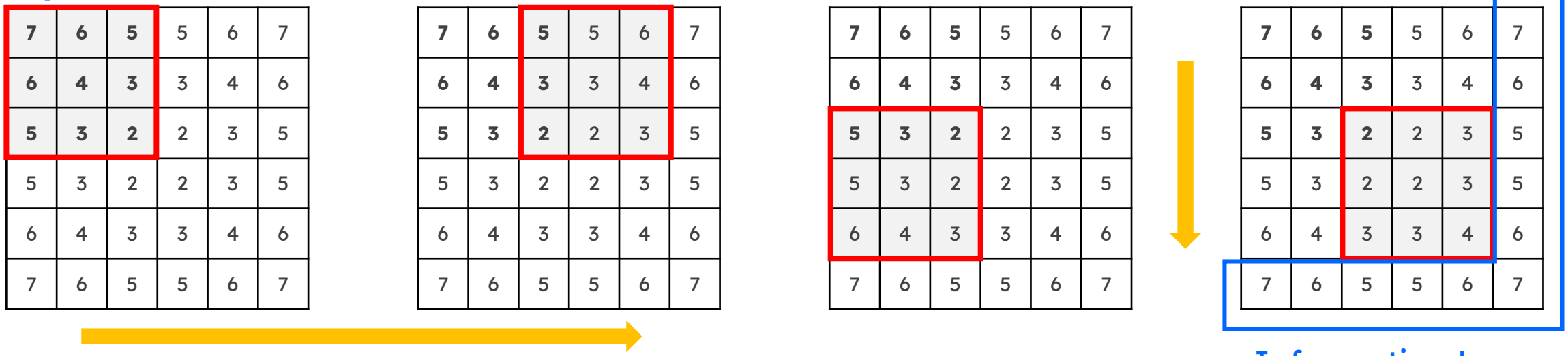
## Padding Demo

Given stride=2 and Padding=0, …

**Filter**

| | | |
|---|---|---|
| 0 | -1 | 0 |
| -1 | 5 | -1 |
| 0 | -1 | 0 |

**Input**



# Strides & Padding

## Padding Demo

Given stride=2 and Padding=1, …

**Filter**

| | | |
|---|---|---|
| 0 | -1 | 0 |
| -1 | 5 | -1 |
| 0 | -1 | 0 |

**Input**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **0** | **0** | **0** | 0 | 0 | 0 | 0 | 0 |
| **0** | **7** | **6** | 5 | 5 | 6 | 7 | 0 |
| **0** | **6** | **4** | 3 | 3 | 4 | 6 | 0 |
| 0 | 5 | 3 | 2 | 2 | 3 | 5 | 0 |
| 0 | 5 | 3 | 2 | 2 | 3 | 5 | 0 |
| 0 | 6 | 4 | 3 | 3 | 4 | 6 | 0 |
| 0 | 7 | 6 | 5 | 5 | 6 | 7 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 0 | **0** | **0** | **0** | 0 | 0 | 0 |
| 0 | 7 | **6** | **5** | **5** | 6 | 7 | 0 |
| 0 | 6 | **4** | **3** | **3** | 4 | 6 | 0 |
| 0 | 5 | 3 | 2 | 2 | 3 | 5 | 0 |
| 0 | 5 | 3 | 2 | 2 | 3 | 5 | 0 |
| 0 | 6 | 4 | 3 | 3 | 4 | 6 | 0 |
| 0 | 7 | 6 | 5 | 5 | 6 | 7 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | **0** | **0** | **0** | 0 |
| 0 | 7 | 6 | 5 | **5** | **6** | **7** | 0 |
| 0 | 6 | 4 | 3 | **3** | **4** | **6** | 0 |
| 0 | 5 | 3 | 2 | 2 | 3 | 5 | 0 |
| 0 | 5 | 3 | 2 | 2 | 3 | 5 | 0 |
| 0 | 6 | 4 | 3 | 3 | 4 | 6 | 0 |
| 0 | 7 | 6 | 5 | 5 | 6 | 7 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

# Strides & Padding

## Padding Demo

Given stride=2 and Padding=1, …

**Filter**

| | | |
|---|---|---|
| 0 | -1 | 0 |
| -1 | 5 | -1 |
| 0 | -1 | 0 |

**Input**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | **0** | **0** | **0** | 0 |
| 0 | 7 | 6 | 5 | **5** | **6** | **7** | 0 |
| 0 | 6 | 4 | 3 | **3** | **4** | **6** | 0 |
| 0 | 5 | 3 | 2 | 2 | 3 | 5 | 0 |
| 0 | 5 | 3 | 2 | 2 | 3 | 5 | 0 |
| 0 | 6 | 4 | 3 | 3 | 4 | 6 | 0 |
| 0 | 7 | 6 | 5 | 5 | 6 | 7 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |



| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 7 | 6 | 5 | 5 | 6 | 7 | 0 |
| **0** | **6** | **4** | 3 | 3 | 4 | 6 | 0 |
| **0** | **5** | **3** | 2 | 2 | 3 | 5 | 0 |
| **0** | **5** | **3** | 2 | 2 | 3 | 5 | 0 |
| 0 | 6 | 4 | 3 | 3 | 4 | 6 | 0 |
| 0 | 7 | 6 | 5 | 5 | 6 | 7 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |



| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 7 | 6 | 5 | 5 | 6 | 7 | 0 |
| 0 | 6 | **4** | **3** | **3** | 4 | 6 | 0 |
| 0 | 5 | **3** | **2** | **2** | 3 | 5 | 0 |
| 0 | 5 | **3** | **2** | **2** | 3 | 5 | 0 |
| 0 | 6 | 4 | 3 | 3 | 4 | 6 | 0 |
| 0 | 7 | 6 | 5 | 5 | 6 | 7 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

# Elements in CNN

| Term | Core Concept & Function |
| --- | --- |
| **Kernel** | A small 2D matrix responsible for element-wise operations to capture local features. |
| **Filter** | A collection of Kernels (matching the input channels) that outputs a single Feature Map. |
| **Stride** | The step size/speed at which the Kernel moves, controlling the output size. |
| **Padding** | Border zero-filling used to control output dimensions and protect edge information. |

# Pooling

**Do we need to preserve all pixels in the model?**

Most pixels are unnecessary to the model; therefore, we should keep the less number of pixels and important pixels to describe the patterns of original image.

Here, we introduce a method, called “Pooling”, which only keeps a single value of a region via max or average.

This method could efficiently reduce the dimension of feature map and increase the computational efficiency during model training and inference.

# Pooling

**Pooling** uses a sliding window (e.g., $2 \times 2$) over the Feature Map. The two most common types are:

**Max Pooling**

Captures the most prominent features (such as the brightest points or sharpest edges in an image). It is currently the most widely used pooling method.

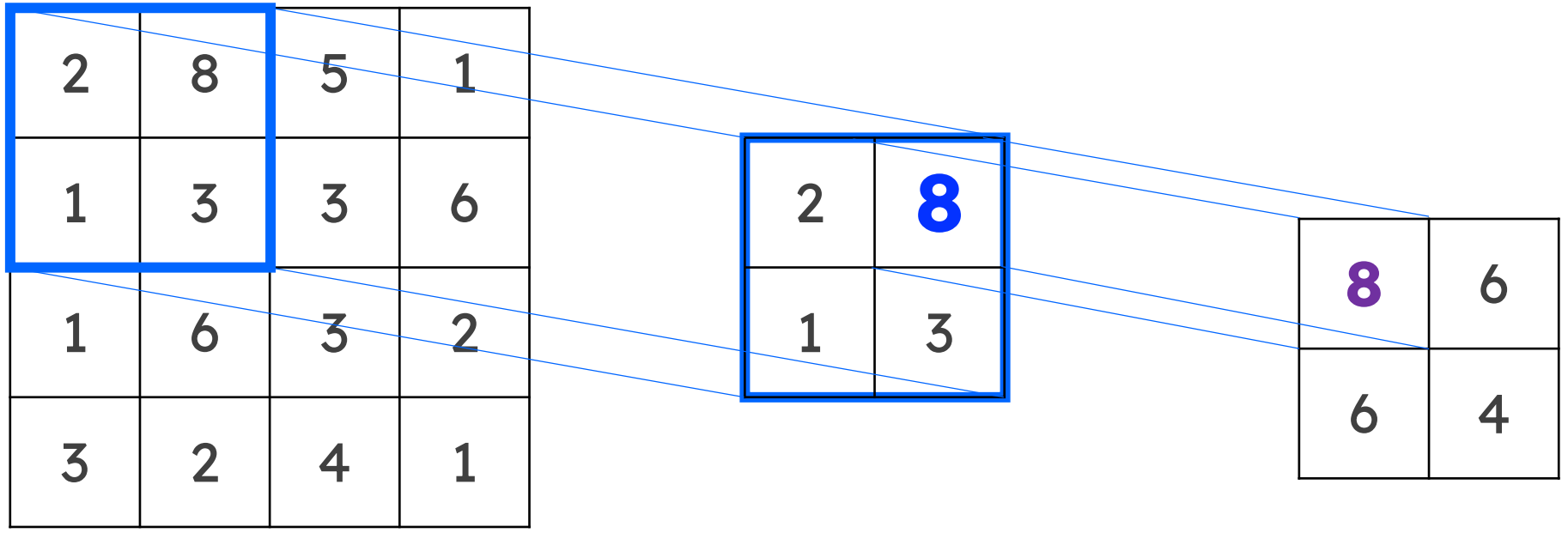
**Average Pooling**

Preserves background or overall distribution information, though Max Pooling generally performs better in most tasks.

# Pooling

## Pooling Demo

Given $2 \times 2$ max pooling with stride=2 and padding=0, ...



# Pooling

**Reduces Computational Cost and Memory Usage**
Pooling reduces the spatial dimensions of feature maps, thereby lowering memory requirements and computational costs in subsequent layers. It may also reduce the number of trainable parameters when followed by fully connected layers.

**Facilitates Receptive Field Expansion**
By downsampling feature maps, pooling allows subsequent layers to aggregate information from larger regions of the original image, supporting the learning of broader spatial patterns.

# Pooling

**Improves Robustness to Small Spatial Translations**
Max pooling can preserve strong feature responses despite small shifts within a pooling window. However, it does not guarantee invariance to translation, rotation, or scaling.

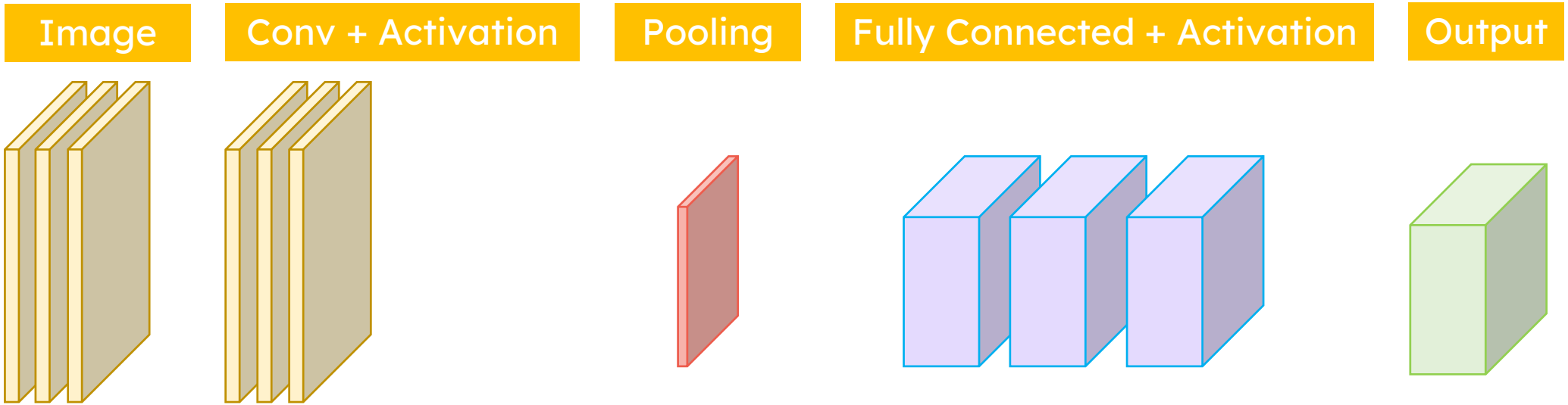
$$\begin{bmatrix} 5 & 2 \\ 8 & 1 \end{bmatrix} \rightarrow 8 \qquad \begin{bmatrix} 2 & 8 \\ 1 & 5 \end{bmatrix} \rightarrow 8$$

**May Improve Generalization**
By reducing spatial detail and sensitivity to small local variations, pooling can act as a form of implicit regularization in some architectures. However, its effect on overfitting is task-dependent and not guaranteed.

# CNN Architect

As we introduce all elements in CNN, now, we are about to tell you how to construct a CNN model.

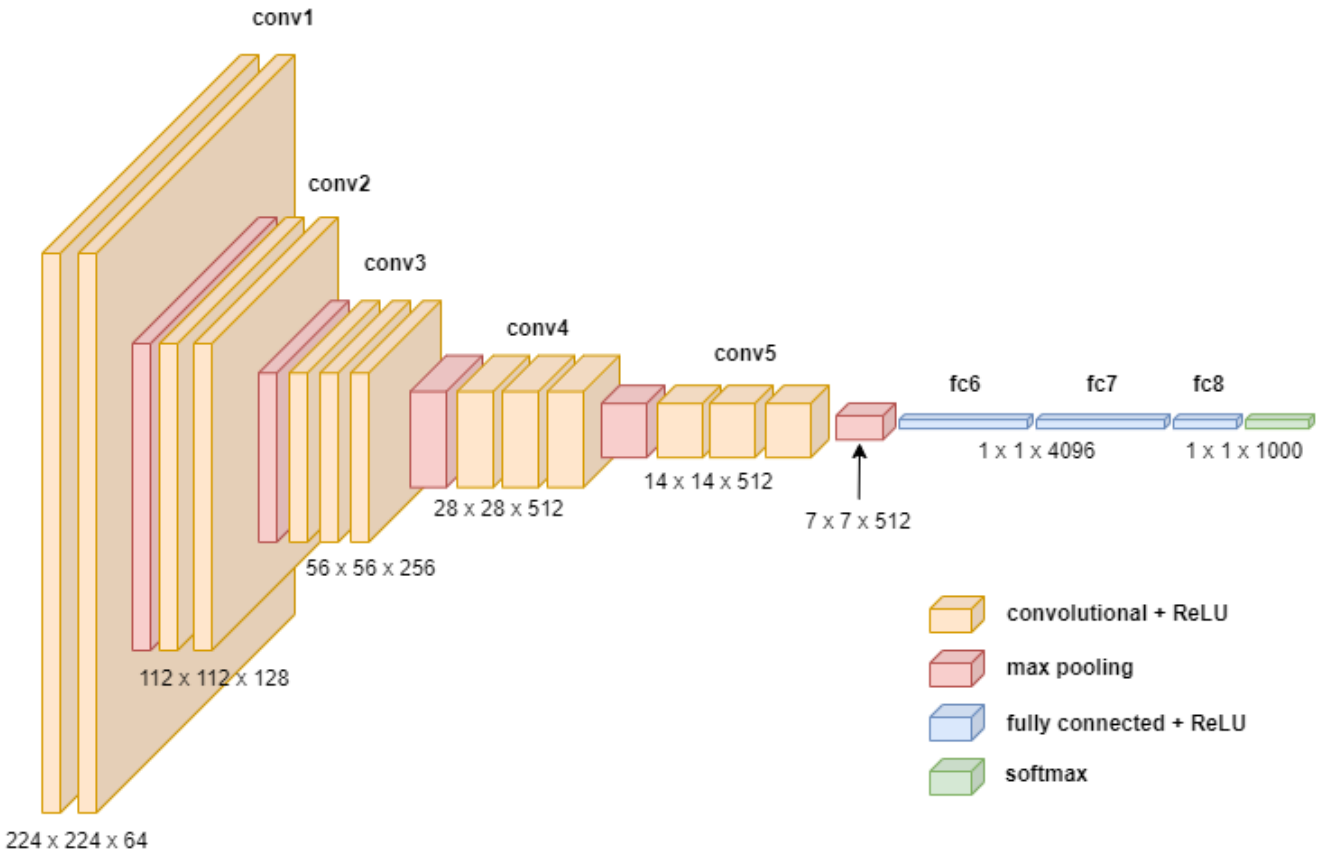
Typically, most models follows …



# CNN Architect

**For example, …**

VGG16 architecture




Source: https://towardsdatascience.com/how-to-easily-draw-neural-network-architecture-diagrams-a6b6138ed875/

# CNN Architect

**Convolution Layer**

$$X \to W * X + b, W \; is \; learned \; filters$$

**Activation Function**

e.g., ReLU

$$f(x) = \max(0, x)$$

Add non-linearity term; otherwise, this model is linear transformation by multiple linear operation.

**Pooling**

Spatial downsampling to reduce the spatial resolution of feature map

# CNN Architect



$4 \times 4 \times 512$ $\quad$ $8{,}192{,}000$

## Classification Head :: Flatten

$$4 \times 4 \times 512 \rightarrow \boldsymbol{Classification\ Head} \rightarrow \begin{bmatrix} z_{cat} \\ z_{dog} \\ z_{rat} \end{bmatrix}$$

The functionality of classification head is to transform the feature values to probability.

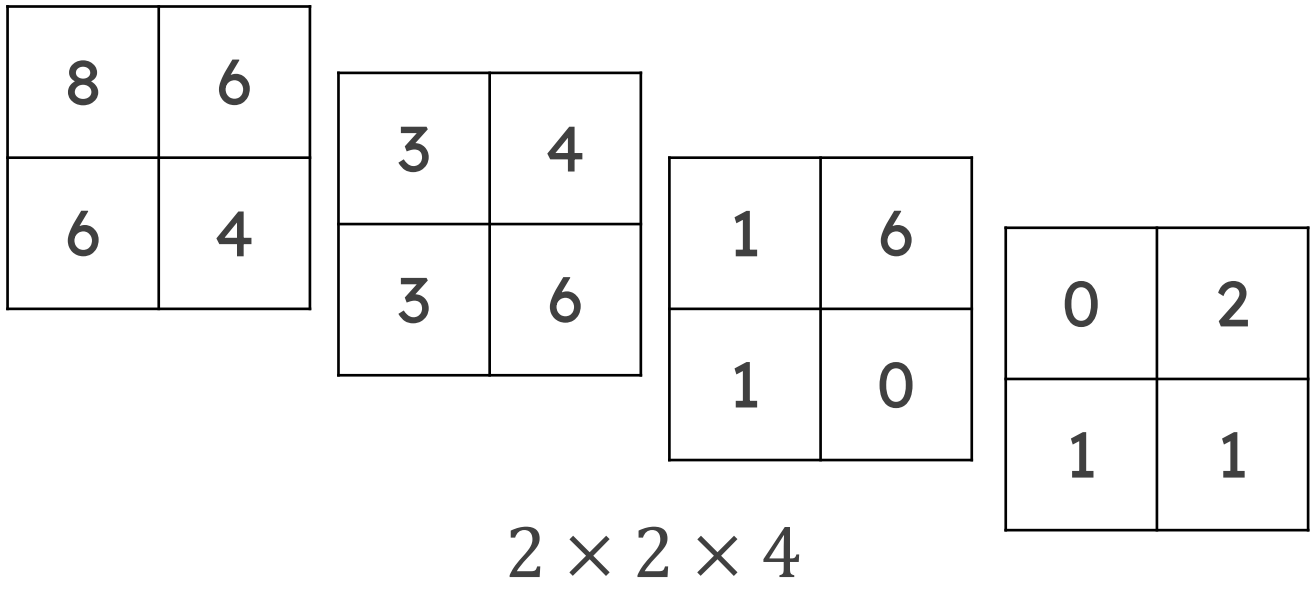
$$\boldsymbol{flatten} \rightarrow Fully\ Connected \rightarrow Softmax$$

If we have a $4 \times 4 \times 512$ and $1{,}000$ neurons, then we have to train 8,192,000 parameters, which is a huge number of trainable parameters.

# CNN Architect

**Classification Head :: Global Average Pooling (GAP)**

GAP is the method to compute all average spatial position from each feature map.




$$g_c = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} F_{i,j,c}$$

# CNN Architect

**Classification Head :: Fully Connected (FC) Layer**

Assuming the feature vector after Global Average Pooling (GAP) is $\mathrm{x} \in \mathbb{R}^{512}$, our goal is to perform a 3-class classification for Cat, Dog, and Bird. Therefore, the final layer consists of 3 output neurons.

In the FC layer, the linear transformation is …

$$z = Wx + b$$

where the weight matrix is $W \in \mathbb{R}^{3\times512}$ and the bias vector is $b \in \mathbb{R}^3$.

# CNN Architect

**Classification Head :: Softmax**

For a multi-class and mutually exclusive classification problems:

$$P(y = k|x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_k}}$$

Classification Prediction

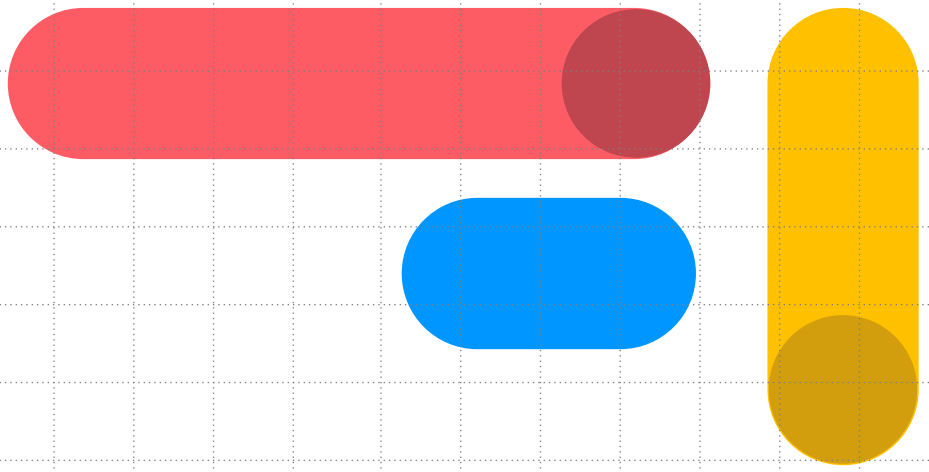
$$\hat{y} = arg\max_{k} P(y = k|x)$$

During the training process, we adopt cross-entropy loss

$$L = -\sum_{k=1}^{K} y_k \log(\hat{p}_k)$$

# References


1) Stanford. What-is-computer-vision
2) Zou, L. (2022). Meta-learning: theory, algorithms and applications. Elsevier.
3) Suri, Abhinav, et al. "Vertebral deformity measurements at MRI, CT, and radiography using deep learning." Radiology: Artificial Intelligence 4.1 (2021): e210015.
4) Zou, L. (2022). *Meta-learning: theory, algorithms and applications*. Elsevier.



# The End

## Thank you for your attention!

Email: chchan@ntnu.edu.tw

Website: https://toodou.github.io/